CS324 Distributed Systems

Assignment 2

Part 1: Report

gRPC worker cluster with leader election, failure recovery and streamed progress

Course	CS324 Distributed Systems
Group	**[Group number / tutorial group]**
Repository	gRPC-Leader-Election
Date	20 October 2026

This report describes the design, implementation and verification of a distributed worker cluster in which every inter-process operation uses gRPC and Protocol Buffers. It covers the election protocol, the reasons for choosing gRPC over Java RMI, failure detection, coordinator state recovery, and the alternatives considered. The system preserves the Assignment 1 worker behaviours while adding real-time progress streaming and both worker and coordinator failure tolerance.

Group member names, student IDs and contribution percentages are highlighted in yellow. Replace those placeholders before submission; the percentages must sum to 100%. Meeting-minute details are also highlighted in Appendix C.

Group members and contributions

Member	Student ID	Main responsibilities	Contribution
[Member 1 Name]	**[Student ID]**	Election, quorum, integration, report	**[25%]**
[Member 2 Name]	**[Student ID]**	Coordinator failover, checkpoint replication, recovery	**[25%]**
[Member 3 Name]	**[Student ID]**	Job distribution, computation, progress streaming, client UI	**[25%]**

[Member 4 Name]	**[Student ID]**	Bootstrap, gRPC contracts, tests, build/CI, documentation	**[25%]**
Total			100%

Q1. Election protocol and its gRPC interface

The cluster elects one coordinator without a fixed leader or a separate election server. Each worker is both a client and a server of ElectionService, defined in shared/src/main/proto/election.proto. The service deliberately contains only three RPCs: SendElection carries one wave hop, SendEcho carries one subtree answer back toward the origin, and AnnounceCoordinator publishes the winner. Every call returns a small acknowledgement; a failed hop is a signal to prune that edge and repair the topology rather than a reason to retry forever.

Service contract

The frozen contract below is the only interface used for election traffic. Generated Java stubs are shared by bootstrap, worker, client and integration tests.

```
syntax = "proto3";
package distrilab;
option java_package = "com.cs324.election.proto";
option java_multiple_files = true;


service ElectionService {
  rpc SendElection(ElectionMessage) returns (ElectionAck);
  rpc SendEcho(EchoMessage) returns (EchoAck);
  rpc AnnounceCoordinator(CoordinatorAnnouncement) returns (CoordinatorAck);
}


message ElectionMessage {
  int32 origin_id = 1;
  int32 candidate_id = 2;
  int32 candidate_jac = 3;
  int64 message_id = 4;
  int64 term = 5;
  repeated int32 voters = 6;
}
message ElectionAck {}


message EchoMessage {
  int64 message_id = 1;
  int32 from_worker_id = 2;
  int32 best_candidate_id = 3;
  int32 best_candidate_jac = 4;
  int64 term = 5;
  repeated int32 voters = 6;
}
message EchoAck {}


message CoordinatorAnnouncement {
  int32 coordinator_id = 1;
  int64 message_id = 2;
  int64 term = 3;
}
message CoordinatorAck {}
```

message_id is generated from the sender identity and a local counter, so two roots cannot accidentally create the same wave. term is the leadership generation. voters is the union of workers that participated, and is used for the majority check before an announcement is allowed.

Method responsibilities

SendElection. The root votes for itself and sends its candidate to every neighbour. A receiver compares (JAC, worker ID), forwards the winning candidate to every neighbour except its parent, and records the voters it has seen. A duplicate message_id is not silently dropped: the receiver echoes the best candidate it already knows, so a cycle cannot leave a parent waiting forever.

SendEcho. When all pending children have answered, a node merges their best candidates and voter sets with its own and sends one echo to its parent. A per-wave lock prevents a duplicate echo from completing the same wave twice. The root therefore learns the winner and the full participating set without all-to-all communication.

AnnounceCoordinator. The root announces only after the voter union is a strict majority of ready workers known through the bootstrap directory. Otherwise it records ELECTION_NO_QUORUM and announces nobody. The announcement carries a newer term; a worker accepts the (term, coordinator ID) pair once and rejects stale or termless announcements.

Election rule

- A lower job-allocation counter (JAC) wins; when JACs are equal, the higher worker ID wins. This preserves the Assignment 1 load-balancing metric instead of electing by ID alone.
- A root that cannot collect a strict majority never announces a coordinator, so a minority partition cannot install a leader by itself.
- After the winner is known, QuorumMonitor probes ready workers every second with a 600 ms deadline, renews a 2.5-second lease, and forces a step-down after two failed checks. A leader without a majority cannot accept new work, preventing split brain.
- Terms increase with each election and are attached to announcements, so a deposed coordinator cannot reclaim leadership with an old message.

Why an echo wave

The unstructured network may be a chain, a triangle or a larger mesh, and edges can disappear while a wave is in flight. The echo wave converges on any connected graph, uses only local knowledge, and needs no global clock or ring order. It also lets every node contribute its JAC. The cost is O(E) messages per wave, acceptable at assignment scale, and the same exchange supplies the voter set used for the majority check.

Q2. gRPC versus Java RMI

The implementation uses gRPC for every distributed operation: registry discovery, worker control, election, job submission and assignment, checkpoint replication and progress streaming. No production class imports java.rmi, extends Remote or exports an RMI object, and NoRmiDependencyTest enforces that property. The comparison below is therefore not theoretical: it explains the transport actually used.

Concern	gRPC + Protocol Buffers	Java RMI
Contract	Language-neutral .proto schema; generated stubs for many languages.	Java interfaces; Java-only clients.
Serialization	Explicit protobuf fields with stable field numbers; inspectable payloads.	Java object serialization; brittle across class changes and harder to inspect.
Streaming	First-class unary, client-streaming, server-streaming and bidirectional RPCs.	No equivalent streaming abstraction; callbacks or polling are ad hoc.
Failure semantics	Deadlines, cancellation and typed status codes (UNAVAILABLE, DEADLINE_EXCEEDED, ...).	RemoteException and timeouts; less uniform cross-language semantics.
Transport	HTTP/2 multiplexing over one connection; flow control and backpressure.	JRMP, a Java-specific protocol; often one request per connection.
Security	TLS and token/metadata interceptors are standard; plaintext is an explicit option.	Security requires separate configuration and is commonly omitted in demos.
Discovery	Bootstrap RegistryService returns typed WorkerEndpoint messages.	RMI registry or custom lookup; registry is another Java-specific component.
Build cost	protoc + gRPC code generation adds a build step.	No code generation, but no schema contract either.

Why gRPC is the better fit here

First, the assignment requires progress to be pushed to the client as work actually completes. JobProgressService.WatchJob is a server stream of typed JobProgress messages, and the client can reconnect with the last known job ID. Re implementing that in RMI would mean a callback object, manual listener registration and custom failure handling. Second, the cluster needs explicit per-call deadlines: a dead worker must not stall a heartbeat, a quorum probe or a task assignment. gRPC deadlines and status codes make those decisions local and testable. Third, the six proto files form a frozen, reviewable contract shared by bootstrap, worker, client and integration tests. That is valuable in a team project because a field-number change is visible in review, while a Java serialization change can silently break an older process.

The cost is real: the Maven build must run protoc and the gRPC plugin, and generated sources have to be on the IDE classpath. The project handles this in the shared module and documents the module classpath in the README. The benefit is a transport with streaming, deadlines, flow control and a language-neutral contract, which maps directly to the Assignment 2 requirements.

RMI would have been shorter to start but would have made streaming and cross-process failure handling more custom and more Java-specific.

Q3. Detecting failed workers and failed coordinators

Failure detection is deliberately conservative: a slow reply is not treated as a crash, but prolonged silence eventually removes the process from the set of voters. The same principle is used at two levels: the coordinator monitors its own majority, and every worker monitors the coordinator it currently believes in.

Coordinator health

CoordinatorHealthMonitor sends a bounded FailoverService.Heartbeat to the current coordinator. CoordinatorHealth classifies each result without performing I/O:

State	Condition	Response
ALIVE	Successful heartbeat within the 400 ms slow threshold.	Reset the miss counter; keep serving and monitoring.
SUSPECT	Successful but slow heartbeat, or one or two consecutive misses.	Keep monitoring; a slow peer is demonstrably up, so no election starts.
FAILED	Three consecutive misses, or no success for eight seconds.	FailureGate admits one failover request for this failure window, then survivors elect.

The per-call deadline is 600 ms and the interval is 1000 ms, so a probe cannot overlap the next one indefinitely. A timeout is only one miss; the threshold of three tolerates a GC pause or a transient scheduler delay. The eight-second silence rule prevents an earlier success from masking a peer that has stopped answering mid-window. These constants are in Constants.java and are covered by CoordinatorHealthTest and CoordinatorFailoverTest.

A coordinator also has to prove that it still has a majority. QuorumMonitor probes each ready worker's WorkerControlService.GetStatus every second with a 600 ms deadline. Self and each reachable member count as a vote; the threshold is floor(n/2)+1. A successful check renews a 2.5-second lease. Two consecutive failed checks clear the coordinator view and record QUORUM_LOST and LEADERSHIP_STEP_DOWN. New admissions require a fresh lease, so a leader that has just lost contact cannot keep serving while a replacement is elected.

Worker failure during a job

The coordinator probes outstanding assignments every 500 ms with a one-second timeout. If a worker is unreachable, its unfinished slices are reassigned through fresh discovery, including to a late-joining worker. Each slice has a stable task ID and an attempt number; the assignment key is (job ID, task ID, attempt). A stale result from the old owner cannot complete a replacement attempt, and a completed slice is retained and included once in the aggregate. A failed worker can therefore lose work without failing the job. The progress stream preserves units that the worker had already completed and changes only that worker's status to Failed.

Slow versus failed

A successful but slow heartbeat is a SUSPECT signal, not a failure. The distinction matters because a false failure triggers an unnecessary election and can split work. The failure threshold and the quorum lease are intentionally longer than one RPC deadline, but short enough to demonstrate recovery in seconds. A genuinely crashed process stops answering entirely and eventually crosses both the miss threshold and the silence bound. TCP errors are treated as failures immediately; a timeout is treated as a miss until the threshold is reached. The tests exercise both slow and dead coordinators, minority partitions and leader step-down.

Q4. Coordinator failure and state recovery

A coordinator is not allowed to acknowledge a job until at least one peer has confirmed the acceptance snapshot. The snapshot contains the original SubmitJobRequest, the client callback endpoint, the acceptance time, and every slice with its input, owner and attempt. The coordinator sends it to published peers over FailoverService.Replicate and waits up to one second for an acknowledgement whose sequence covers the snapshot. No peer, a timeout or an RPC error is not confirmation: the submission returns retryable UNAVAILABLE, no computation starts, and the reserved job-budget slot is released. At least two live workers are therefore needed to accept new work.

State recovered after a takeover

State	What the checkpoint keeps	Why it matters
Job record	Job ID, computation type, original request and client endpoint.	Rebuild the submission without asking the client to resend it.
Task slices	Task ID, payload, assigned worker and attempt number.	Preserve completed work and identify what must be reassigned.
Execution result	Per-slice result and the attempt that produced it.	Retain useful work and reject late results from an older attempt.
Job outcome	Final aggregate and a client-delivery flag.	Avoid recomputation and resend a result whose callback was not acknowledged.
Timing	Original acceptance timestamp and remaining budget.	Reconstruct the deadline, with a minimum recovery allowance.
Ordering metadata	Per-job sequence, leadership term and coordinator ID.	Fold updates idempotently and reject stale or duplicate state.

Replicated checkpoint log

Each checkpoint entry has a job ID, a per-job sequence, the producing term and coordinator ID, and one of three payloads: JobAccepted, SliceUpdated or JobFinished. Per-job sequence numbers make folding idempotent: a duplicate or out-of-order entry is ignored instead of applied twice. Every outbound push carries a cumulative snapshot, so an older asynchronous RPC cannot overwrite a newer view. A JobFinished entry is a tombstone: it carries the final result (not one slice's result) and a delivered flag, so a recovered job is not rebuilt and a lost client callback can be replayed. Finished state has a bounded retention window so a long-lived replica does not grow forever.

The checkpoint log is in memory on every worker. Acceptance is synchronous with one peer acknowledgement; later slice, completion and delivery updates are asynchronous. That is a deliberate trade-off between latency and durability. The system is not a disk-backed Raft/Paxos log: if every replica is lost, or the whole cluster is restarted, the checkpoints are lost. The report states this limitation rather than claiming exactly-once durability.

Takeover

When a worker becomes coordinator, it temporarily refuses new jobs. It fetches checkpoint snapshots from every reachable peer and merges them per job, keeping the newest sequence for each job. It does not steal a job that is owned by a live coordinator. For a job whose owner failed, it preserves completed slices and resumes unfinished slices with fresh discovery. The restored job keeps its original deadline, subject to a minimum recovery window so a job that was nearly out of time still gets a fair chance. If every slice is already complete, the job aggregates immediately instead of waiting for a timeout. If the job had already finished, the saved aggregate is delivered; delivery is recorded only after the client callback acknowledges it.

Recovery has two explicit limits. First, it needs a surviving replica and a reachable replacement coordinator; a two-worker cluster cannot recover from the coordinator's failure because there is no majority left. Second, callback delivery is at-least-once in the presence of lost acknowledgements, so the client deduplicates by job ID. These limits are documented in docs/failover.md and are visible in the process-level GrpcOnlyClusterTest, which terminates a real coordinator and checks the recovered answer and completed progress.

Bootstrap state and membership

The bootstrap is still a single directory with no failover replica, but it no longer retains crashed workers forever. Workers refresh their membership entry every five seconds, and the bootstrap reaps an entry after fifteen seconds of silence. That keeps the majority denominator honest. Worker and job ID counters are persisted in bootstrap-state.properties (override with --state-file) so a bootstrap restart does not immediately reuse an identity. Active membership is not persisted: after a bootstrap restart, workers must re-register, and the documented safe procedure is to restart the whole cluster if the old membership is gone.

Q5. Overall failure detection and recovery strategies

DistriLab combines five mechanisms because each addresses a different failure: unreachability, conflicting leadership, lost work, stale messages and a failed client callback. The design targets a small, unreliable, unstructured worker network and must recover without manual restart.

Selected strategies and justification

Heartbeat monitoring. Followers probe the coordinator every 1,000 ms with a 600 ms deadline. A successful response under 400 ms is ALIVE; a slower response or one or two misses is SUSPECT; three misses or eight seconds without success is FAILED. This avoids an election after a single delayed reply, while still detecting a process that has stopped.

Echo-wave election. Election messages follow the current neighbour graph and echoes merge each subtree's best candidate and voters. The lowest JAC wins, with the highest worker ID as tie-breaker. The initiator announces only with a strict majority of ready workers, which suits an unstructured overlay better than an ID-ordered Bully or ring.

Majority-quorum protection. QuorumMonitor probes ready workers every second, uses a 600 ms deadline, clears the view after two failed checks and renews a 2,500 ms lease. New jobs require a fresh lease. A leader in a minority partition therefore stops accepting work instead of creating conflicting results, prioritising safety over availability.

Replicated checkpoints. JobAccepted, SliceUpdated and JobFinished entries are replicated to peers with per-job sequence numbers and terms. At least one peer must confirm acceptance before the client is told the job was accepted. Later updates are asynchronous, which keeps submission responsive but means the newest intermediate update may be lost.

Task reassignment and duplicate control. When an owner is unreachable, its unfinished slices are reassigned with an incremented attempt. Stable task IDs and checkpointed results preserve completed units and reject a late result from an older attempt. This is cheaper than restarting the whole job and is safer than accepting the first reply blindly.

Alternatives considered

Alternative	Attraction	Why the chosen design differs
Fixed central coordinator	No election and simple testing.	It is a single point of failure and fails the leader-election requirement. The bootstrap remains only a directory.
Bully or ring election	Deterministic and low message count on an ordered ring.	An unstructured, healing mesh has no stable ring, and ID order ignores the JAC load metric.
Gossip membership	Scales and tolerates churn.	Convergence is slower for a demo and still needs a majority rule to avoid split brain.
Full Raft or Paxos	Durable, strongly consistent	Much greater implementation and test complexity; the

	replicated log.	project uses a majority lease and replicated checkpoints, not a consensus library.
Polling for progress	No streaming API.	It is not real-time and causes stale bars or heavy polling; gRPC server streaming fits the requirement.

Recovery sequence

1. Detect an unresponsive worker or coordinator through gRPC deadlines, refused connections and the health state machine.
2. For a coordinator failure, start the echo wave and require strict-majority participation before announcing a replacement.
3. Install the new coordinator with a higher term; reject stale announcements and step down a leader that loses its quorum lease.
4. Fetch checkpoint snapshots from reachable peers, merge by job and sequence, and preserve completed slices.
5. Reassign unfinished slices from unreachable owners with new attempts; resume progress reporting from recovered state.
6. Redeliver a completed result whose client callback was not acknowledged. Delivery is at-least-once, so the client deduplicates by job ID.

Limitations and conclusion

A timeout proves unreachability, not a crash: a slow or isolated process can look failed. Asynchronous replication can omit the last intermediate checkpoint, and a cluster without a majority deliberately rejects new work. Checkpoints are in memory, so losing every replica loses recovery state. The bootstrap is a single directory, and callbacks are at-least-once. These limits are documented in docs/failover.md and are preferable to claiming durability the implementation does not provide.

The combination is appropriate for the assignment: it elects on an arbitrary graph, prevents a minority leader from serving, preserves accepted work across a coordinator change and recovers unfinished slices without restarting every job. Full Raft would give stronger durability but would replace the assignment's focus with a consensus implementation. Bounded retention for jobs, tasks, election waves and finished checkpoints keeps the in-memory design stable during a long demo.

References

Google. (n.d.). gRPC documentation. https://grpc.io/docs/

Google. (n.d.). Protocol Buffers documentation. https://protobuf.dev/

Oracle. (n.d.). Java Remote Method Invocation (Java RMI). https://docs.oracle.com/javase/8/docs/technotes/guides/rmi/

Van Steen, M., & Tanenbaum, A. S. (2025). Distributed systems (4th ed., Version 4.03). Sections 4.2 and 5.4; Chapters 7 and 8.

Appendix A. Git history

Generated from the repository with git log --graph --date=short --pretty=format:'%h %ad %an%x09%s'. The initial commit imports the Assignment 1 code, and later commits add the gRPC migration, failover, quorum and the Assignment 2 hardening.

```
* 52739d0 2026-10-20 Surbhit Nand  Add submission-ready A2 Word report package
* 1a532c8 2026-10-20 Surbhit Nand  Document operational limits, leases and verification
* 9e88cec 2026-10-20 Surbhit Nand  Add CI, coverage and scale checks
* 15fa1fb 2026-10-20 Surbhit Nand  Bound coordinator retention and share gRPC channels
* 6eb9225 2026-10-20 Surbhit Nand  Harden bootstrap membership leases and persist ID counters
* bb2090e 2026-10-20 Alvin Kumar   feat(grpc): modularize cluster and enforce quorum leadership
*   ef19cda 2026-10-20 Surbhit Nand        Merge origin/main into main
|\
| *   a394a6f 2026-10-19 Alvin Kumar       Merge pull request #2 from Surbhit-Nand/test-branch
| |\
| | * 1967d87 2026-10-19 Alvin Kumar       fix(progress): reconnect streams and rediscover recovered jobs
| | * 96735e5 2026-10-19 Alvin Kumar       fix(failover): replicate cumulative state across reordered RPCs
| | * d120f58 2026-10-19 Alvin Kumar       fix(failover): share delivered tombstones during takeover
| | * be5d6b5 2026-10-19 Alvin Kumar       fix(failover): preserve checkpoint sequences when taking snapshots
| | * 7beb1b7 2026-10-19 Alvin Kumar       fix(grpc): migrate executable peer links and monitoring off RMI
| | * c192621 2026-10-19 Alvin Kumar       fix(progress): retain completed units when remaining work fails
| | * b04be46 2026-10-19 Alvin Kumar       fix(failover): acknowledge replay only after client delivery
| | * 668da09 2026-10-19 Alvin Kumar       fix(failover): replay the saved final job result
| | * 59164a3 2026-10-19 Alvin Kumar       fix(failover): finish recovered jobs with completed slices
| | * c441442 2026-10-18 Aryan Sewak       Fix two defects found reviewing the failover, and test the rules
directly
| | * 4924f4f 2026-10-18 Aryan Sewak       Name the remaining failover event kinds and refresh the test count
| | * 5dd93d8 2026-10-18 Aryan Sewak       Document election over gRPC and coordinator failover
| | * f3fa15b 2026-10-18 Aryan Sewak       Test coordinator failover end to end and harden what it exposed
| | * 92fa42a 2026-10-18 Aryan Sewak       Wire failover into the worker: replicate, detect, take over
| | * 58a4bda 2026-10-18 Aryan Sewak       Give failover a replication seam, a restore entry point and
reassignment fix
| | * 9474df3 2026-10-18 Aryan Sewak       Detect coordinator failure with an alive/suspect/failed rule
| | * 5f6a3ca 2026-10-18 Aryan Sewak       Add the failover contract and the replicated job log
| | * 4c07dce 2026-10-18 Aryan Sewak       Cover the gRPC election with real TCP tests and document it
| | * be400c8 2026-10-18 Aryan Sewak       Run the election wave over gRPC behind a transport seam
| | * ad31f2c 2026-10-18 Aryan Sewak       Add additive term fields to the election contract
| |/
* | f96a490 2026-10-18 Surbhit Nand        Register generated protobuf sources with build-helper for m2e
* | 1607bc1 2026-10-18 Surbhit Nand        Resolve os.detected.classifier in m2e via os-maven-plugin lifecycle
binding
* | fca5ba7 2026-10-18 Surbhit Nand        Bump gRPC to 1.85.0 and drop unreachable catch in BootstrapGrpcService
|/
* 90457c2 2026-10-17 Whynot-hub    Document worker failure recovery and GUI system log in README
*   4b82e1b 2026-10-17 Whynot-hub  Merge worker-failure-recovery into main
|\
| * 881a8c0 2026-10-17 Whynot-hub  Implement gRPC worker failure recovery and colored GUI system log
|/
*   b1e2463 2026-10-16 Whynot-hub  Merge job distribution gRPC integration into main
|\
| * 5db4812 2026-10-16 Whynot-hub  Integrate job distribution with shared gRPC services and bootstrap discovery
| *   93d45d5 2026-10-16 Whynot-hub        Merge remote-tracking branch 'origin/main' into job-distribution
| |\
| |/
|/|
* | 7fbc9f5 2026-10-16 Whynot-hub  Refresh client workspace styling and add View actions
* |   884a66f 2026-10-16 Surbhit Nand      Merge docs/run-instructions: run.ps1 first in How to Run
|\ \
| * | 57f6acc 2026-10-16 Surbhit Nand      Lead How to Run with run.ps1, mark java commands equivalent
|/ /
* |   8c11cc9 2026-10-16 Surbhit Nand      Merge origin/main (PR #1: bootstrap-grpc with review cleanup)
|\ \
| * \   c41c820 2026-10-16 Surbhit Nand    Merge pull request #1 from Surbhit-Nand/feature/bootstrap-grpc
| |\ \
| | * | fe8c26a 2026-10-16 Surbhit Nand    Cleanup: imports, explicit types, dedupe record construction
* | | |   9865115 2026-10-16 Surbhit Nand  Merge docs/grpc-readme: gRPC contracts, migration status, test count
|\ \ \ \
| |/ / /
|/| | |
| * | | 4236476 2026-10-16 Surbhit Nand    Document gRPC contracts, migration status and test count
```

```
|/ / /
* | | 6b260a7 2026-10-16 Surbhit Nand      Merge feature/bootstrap-grpc: dual-stack Bootstrap discovery over gRPC
|\| |
| * |   fc27309 2026-10-16 Surbhit Nand    Merge main (contracts) into feature/bootstrap-grpc
| |\ \
| |/ /
|/| |
* | |   d17337a 2026-10-16 Surbhit Nand    Merge feature/grpc-contracts: frozen gRPC contracts for registry,
election and jobs
|\ \ \
| | * | e860c8a 2026-10-16 Surbhit Nand    Review cleanup: drop dead local, fix import wrap
| | * | 17b41d5 2026-10-16 Surbhit Nand    Dual-stack Bootstrap discovery over gRPC
| |/ /
| * / aa7ce21 2026-10-16 Surbhit Nand      Freeze gRPC contracts for registry, election and jobs
|/ /
* |   1f3b5c6 2026-10-16 Surbhit Nand      Merge refactor/run-script: launcher and gRPC classpath docs
|\ \
| * | 8a6ff4a 2026-10-16 Surbhit Nand      Add run.ps1 launcher and document the gRPC classpath
|/ /
| * 3812a59 2026-10-16 Whynot-hub  Add independent gRPC job distribution and standalone demo
|/
* 8d1a510 2026-10-15 Whynot-hub    Add gRPC job progress streaming and improve client history and scrolling
*   b0b650f 2026-10-15 Surbhit Nand        Merge refactor/range-warning-1b: warn-but-allow for wide PRIMESUM
ranges up to 1B
|\
| * 9163995 2026-10-15 Surbhit Nand        Warn-but-allow for wide PRIMESUM ranges up to 1B
|/
*   5f60d97 2026-10-15 Surbhit Nand        Merge refactor/cluster-activity-events: cluster activity card with
worker event logs
|\
| * f17f447 2026-10-15 Surbhit Nand        Cluster activity card with worker event logs
|/
*   b8ec1a3 2026-10-15 Surbhit Nand        Merge refactor/worker-auto-assign-rewire: auto worker ids/ports and
partition-healing rewire
|\
| * e097c6e 2026-10-15 Surbhit Nand        Auto-assign worker ids and ports plus partition-healing rewire
|/
* 94d74b7 2026-10-13 Surbhit Nand  Update readme file
* 01cb0ed 2026-10-13 Surbhit Nand  Docs: follow repo rename to gRPC-Leader-Election
* f160800 2026-10-13 Surbhit Nand  Docs: point clone URL to Surbhit-Nand/CS324_A2
* eb43999 2026-10-13 Surbhit Nand  Import leader-election-algorithm as initial commit
* 83b74bc 2026-10-18 Surbhit Nand  Agent host session eb74ae47-3da9-4bfb-bcf9-733bda3bab6f - turn 1
* 8936431 2026-10-18 Surbhit Nand  Agent host session eb74ae47-3da9-4bfb-bcf9-733bda3bab6f - turn 1 start
* 9d76162 2026-10-18 Surbhit Nand  Agent host session eb74ae47-3da9-4bfb-bcf9-733bda3bab6f - baseline checkpoint
```

Appendix B. AI tools and disclosure

AI coding assistants were used during development and for report preparation. They were treated as tools, not as group members: the group owns the design decisions, reviewed the output, and is responsible for the submitted code. No AI-generated change was accepted without a build and test run.

Area	Tool / mode	How it was used	Human verification
Election and gRPC migration	OpenAI Codex via the opencode agent	Read the existing code, proposed the transport seams, wrote/edited classes and tests, ran Maven.	Member reviewed the protocol rules, constants and diffs; green build required.
Coordinator failover	OpenAI Codex via the opencode agent	Implemented checkpoint replication, health monitoring and takeover; wrote end-to-end tests and docs.	Member specified the safety rules and reviewed every merge.
A2 hardening and report	DeepSeek V4.1 Flash via the opencode agent	Audited compliance, added bootstrap leases/persistent counters/bounded retention/channel pooling, CI, coverage and this Word report.	Changes were compiled and the full suite was run; placeholders and contribution data are filled by the group.
Documentation	AI-assisted drafting	Drafted README, protocol notes, failure model and report text from the implementation.	Members checked technical claims against code and test results.

Prompting approach: one detailed work order per area, with the frozen-protobuf rule, required tests and the priority order. The assistant was asked to stop and ask when a decision affected group ownership. Existing authorship and commit history were preserved. The historical logs in docs/ai-assisted-audit.md and docs/ai-usage-member2.md contain more detail, including the bugs found by the AI-generated tests and the human review decisions.

Appendix C. Meeting minutes

The dates, attendees and action owners below are highlighted placeholders. Replace them with the group's actual minutes before submission.

Meeting 1 — [Date]

Attendees: [Attendees]

Decisions: Reviewed Assignment 1 feedback, mapped it to the early refactor branches, split A2 into election, failover, job/progress and bootstrap/test work.

Actions: Create the shared proto contracts; agree on the five-job term and JAC rule; set up the module build.

Meeting 2 — [Date]

Attendees: [Attendees]

Decisions: Reviewed the gRPC migration and the first election tests; agreed that no RMI fallback would remain in production code.

Actions: Finish the echo-wave duplicate handling; add chain/triangle tests; document the transport.

Meeting 3 — [Date]

Attendees: [Attendees]

Decisions: Walked through coordinator failure: heartbeat thresholds, majority lease, acceptance replication and takeover merge rules.

Actions: Implement checkpoint replication and the process-level failure test; record known limits.

Meeting 4 — [Date]

Attendees: [Attendees]

Decisions: Final review of the progress stream, worker reassignment, bootstrap leases and the report package; assigned contribution percentages.

Actions: Run mvn verify, fill group details, check the Git log appendix and submit.

Appendix D. Assignment 1 feedback reconciliation

The repository does not store the verbal feedback from the Assignment 1 demo. The table below reconstructs the feedback themes from the early commits and branches named in the Git history. Confirm the wording with the group's own notes if a different list was given.

A1 feedback theme	Evidence in Git history	How A2 addresses it
Automatically assign worker IDs and ports	e097c6e; refactor/worker-auto-assign-rewire	Bootstrap allocates monotonic IDs; workers bind an available gRPC port and publish it.
Repair the topology after a partition	e097c6e; refactor/worker-auto-assign-rewire	Failed hops prune edges; repair tries remembered two-hop peers, then discovered workers.
Show cluster activity in the client	f17f447; refactor/cluster-activity-events	WorkerControlService.GetStatus returns typed recent events; the client has a cluster activity card.
Warn on very wide PRIMESUM ranges	9163995; refactor/range-warning-1b	RangeWarning warns but allows ranges up to one billion; tests cover the boundary.
Stream real progress instead of a fake bar	8d1a510; origin/progress-bar	JobProgressService.WatchJob streams completed/total units; clicking a bar opens the worker breakdown.
Improve client history and scrolling	7fbc9f5; origin/progress-bar	Job history, log scrolling and view actions were added to the client.
Recover from a worker failure	881a8c0; origin/worker-failure-recovery	Stable task IDs and attempts; unreachable slices are reassigned and completed units are retained.
Use a launcher instead of a long classpath command	8a6ff4a; refactor/run-script	run.ps1 builds and starts bootstrap, workers and clients; the classpath is documented.
Freeze the gRPC contracts	aa7ce21; feature/grpc-contracts	Six proto files with frozen field numbers; ProtoContractsTest checks the shapes.
Elect a coordinator and survive its failure	leader-election, coordinator-failover, feature/grpc-majority-quorum	Echo-wave election, strict-majority lease, checkpoint replication and takeover.

Appendix E. Verification and reproducibility

The complete reactor build and test suite was run from the repository root with JDK 25 and Maven 3.9.x:

```
mvn -B verify
```

Result: BUILD SUCCESS, 179 tests run, 0 failures, 0 errors, 0 skipped. The suite includes actual worker JVMs, a forcibly terminated coordinator, stream reconnects, bootstrap lease expiry, persistent counter restart, bounded retention, the multi-worker splitter scale check and the no-RMI guard. JaCoCo is configured in the root POM, and integration-tests/pom.xml produces the aggregate report because all tests live in that module. The GitHub Actions workflow .github/workflows/verify.yml runs the same command on JDK 25 and uploads Surefire and coverage reports.

Setup and run instructions are in README.md. The normal demo uses one bootstrap, four workers and one client. Progress is visible in the client; stopping a worker shows reassignment, and stopping the coordinator shows election and recovery. The bootstrap state file is git-ignored and should not be committed.